

1. Basic Thread Creation Create two threads that print "Thread 1" and "Thread 2" respectively 10 times each. Use both the Thread class and the Runnable interface to implement this.

```
public class ThreadExample1 extends Thread {

    private String threadName;

    public ThreadExample1(String name) {
        this.threadName = name;
    }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(threadName + " is running: " + i);
        }
    }

    public static void main(String[] args) {
        ThreadExample1 t1 = new ThreadExample1("Thread 1");
        ThreadExample1 t2 = new ThreadExample1("Thread 2");

        t1.start();
        t2.start();
    }
}
```

```
public class ThreadExample2 implements Runnable {

    private String threadName;

    public ThreadExample2(String name) {
        this.threadName = name;
    }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println(threadName + " is running: " + i);
        }
    }

    public static void main(String[] args) {
        Thread t1 = new Thread(new ThreadExample2("Thread 1"));
        Thread t2 = new Thread(new ThreadExample2("Thread 2"));

        t1.start();
        t2.start();
    }
}
```

2. Producer-Consumer Problem

Implement the classic Producer-Consumer Problem using `wait()` and `notify()`. Create a shared buffer (e.g., `List<Integer>`) and have one thread produce items (add to the list) while another consumes them (removes from the list).

```
import java.util.LinkedList;
import java.util.List;

public class ProducerConsumer {
    private List<Integer> buffer = new LinkedList<>();
    private final int LIMIT = 5;
    private final Object lock = new Object();

    public void produce() throws InterruptedException {
        int value = 0;
        while (true) {
            synchronized (lock) {
                while (buffer.size() == LIMIT) {
                    lock.wait();
                }
                buffer.add(value++);
                System.out.println("Produced: " + value);
                lock.notify();
            }
            Thread.sleep(1000); // Simulate some delay
        }
    }

    public void consume() throws InterruptedException {
        while (true) {
            synchronized (lock) {
                while (buffer.isEmpty()) {
                    lock.wait();
                }
                int value = buffer.remove(0);
                System.out.println("Consumed: " + value);
                lock.notify();
            }
            Thread.sleep(1500); // Simulate some delay
        }
    }

    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();

        Thread producerThread = new Thread(() -> {
            try {
                pc.produce();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        Thread consumerThread = new Thread(() -> {
            try {
                pc.consume();
            }
        });
    }
}
```

```

        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });

    producerThread.start();
    consumerThread.start();
}
}

```

3. Thread Synchronization Write a program where multiple threads are incrementing a shared counter. Ensure that the counter is thread-safe using synchronized blocks or ReentrantLock.

```

public class SynchronizedCounter {

    private int counter = 0;

    // Increment the counter in a synchronized block
    public synchronized void increment() {
        counter++;
        System.out.println(Thread.currentThread().getName() + " - Counter: " + counter);
    }

    public static void main(String[] args) {
        SynchronizedCounter counter = new SynchronizedCounter();

        // Create multiple threads to increment the counter
        Runnable task = () -> {
            for (int i = 0; i < 100; i++) {
                counter.increment();
            }
        };

        Thread t1 = new Thread(task, "Thread 1");
        Thread t2 = new Thread(task, "Thread 2");

        t1.start();
        t2.start();
    }
}

```

4. Deadlock Simulation Create a scenario where two threads are causing a deadlock by both trying to lock two shared resources in opposite order. Then, try to resolve the deadlock using either a timeout or by ensuring resources are locked in a specific order.

```
public class DeadlockSimulation {

    // Two resources (locks)
    private final Object lock1 = new Object();
    private final Object lock2 = new Object();

    // Method for Thread 1 that tries to lock resources in one order
    public void method1() {
        synchronized (lock1) {
            System.out.println("Thread 1: Holding lock 1...");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            System.out.println("Thread 1: Waiting for lock 2...");
            synchronized (lock2) {
                System.out.println("Thread 1: Holding lock 1 and lock 2...");
            }
        }
    }

    // Method for Thread 2 that tries to lock resources in the opposite order
    public void method2() {
        synchronized (lock2) {
            System.out.println("Thread 2: Holding lock 2...");
            try { Thread.sleep(100); } catch (InterruptedException e) {}

            System.out.println("Thread 2: Waiting for lock 1...");
            synchronized (lock1) {
                System.out.println("Thread 2: Holding lock 1 and lock 2...");
            }
        }
    }

    public static void main(String[] args) {
        DeadlockSimulation deadlock = new DeadlockSimulation();

        // Thread 1 runs method1
        Thread t1 = new Thread(() -> deadlock.method1());
        // Thread 2 runs method2
        Thread t2 = new Thread(() -> deadlock.method2());

        // Start both threads
        t1.start();
        t2.start();
    }
}
```

5. Read-Write Locks Simulate a scenario where multiple threads are reading from a shared resource (e.g., a list), but only one thread can modify it at a time. Use a `ReentrantReadWriteLock` to manage concurrent reads and exclusive writes.

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ReadWriteLockExample {
    private List<Integer> sharedList = new ArrayList<>();
    private ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public void writeToList(int value) {
        rwLock.writeLock().lock();
        try {
            sharedList.add(value);
            System.out.println(Thread.currentThread().getName() + " wrote: " + value);
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void readFromList() {
        rwLock.readLock().lock();
        try {
            if (!sharedList.isEmpty()) {
                System.out.println(Thread.currentThread().getName() + " reading: " +
sharedList);
            } else {
                System.out.println(Thread.currentThread().getName() + " found the list empty.");
            }
        } finally {
            rwLock.readLock().unlock();
        }
    }

    public static void main(String[] args) {
        ReadWriteLockExample example = new ReadWriteLockExample();

        // Writing thread (only one thread will write at a time)
        Thread writer1 = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                example.writeToList(i);
                try {
                    Thread.sleep(500); // Simulate some delay
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }, "Writer-1");

        // Reading threads (multiple threads can read concurrently)
        Thread reader1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
```

```

        example.readFromList();
    }
    try {
        Thread.sleep(300); // Simulate some delay
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}, "Reader-1");

Thread reader2 = new Thread(() -> {
    for (int i = 0; i < 5; i++) {
        example.readFromList();
    }
    try {
        Thread.sleep(300); // Simulate some delay
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}, "Reader-2");

// Start the threads
writer1.start();
reader1.start();
reader2.start();

// Wait for threads to finish
try {
    writer1.join();
    reader1.join();
    reader2.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}
}
}

```

6. Thread Pool with Executors Write a program using Java's `ExecutorService` to create a fixed thread pool that processes 10 tasks (e.g., printing a message or performing a calculation). Submit tasks to the pool and ensure it terminates properly.

```

import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolExample {

    // A simple task class implementing Runnable interface
    static class Task implements Runnable {
        private final int taskId;

        public Task(int taskId) {
            this.taskId = taskId;
        }

        @Override
        public void run() {

```

```

        System.out.println("Task " + taskId + " is being executed by " +
Thread.currentThread().getName());
        try {
            // Simulating some work with a sleep
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Task " + taskId + " was interrupted.");
        }
    }
}

public static void main(String[] args) {
    // Create a fixed thread pool with 3 threads
    ExecutorService executor = Executors.newFixedThreadPool(3);

    // Submit 10 tasks to the executor
    for (int i = 1; i <= 10; i++) {
        executor.submit(new Task(i));
    }

    // Initiate a graceful shutdown
    executor.shutdown();
    System.out.println("All tasks submitted.");

    try {
        // Wait for all tasks to finish execution
        if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
            System.out.println("Forcing shutdown as tasks did not complete in time.");
            executor.shutdownNow(); // Force shutdown if tasks did not complete in time
        }
    } catch (InterruptedException e) {
        executor.shutdownNow(); // Force shutdown on interruption
    }

    System.out.println("Executor shutdown.");
}
}

```

7. Bank Account Simulation Simulate a bank account system where multiple threads (representing customers) are withdrawing and depositing money concurrently. Ensure that the account balance is thread-safe using synchronization mechanisms.

```
class BankAccount {  
    private double balance;  
  
    public BankAccount(double initialBalance) {  
        this.balance = initialBalance;  
    }  
  
    // Synchronized method to deposit money  
    public synchronized void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println(Thread.currentThread().getName() + " deposited " + amount + ",  
New balance: " + balance);  
        }  
    }  
  
    // Synchronized method to withdraw money  
    public synchronized void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
            System.out.println(Thread.currentThread().getName() + " withdrew " + amount + ", New  
balance: " + balance);  
        } else {  
            System.out.println(Thread.currentThread().getName() + " tried to withdraw " + amount  
+ ", but insufficient balance.");  
        }  
    }  
  
    // Get current balance  
    public double getBalance() {  
        return balance;  
    }  
}  
  
public class BankAccountSimulation {  
    public static void main(String[] args) {  
        // Create a bank account with an initial balance of 1000  
        BankAccount account = new BankAccount(1000);  
  
        // Create threads representing customers  
        Thread customer1 = new Thread(() -> {  
            account.deposit(500);  
            account.withdraw(300);  
        }, "Customer-1");  
  
        Thread customer2 = new Thread(() -> {  
            account.deposit(200);  
            account.withdraw(100);  
        }, "Customer-2");  
  
        Thread customer3 = new Thread(() -> {  
            account.withdraw(700);  
        }, "Customer-3");  
    }  
}
```

```

        account.deposit(400);
    }, "Customer-3");

    // Start all customer threads
    customer1.start();
    customer2.start();
    customer3.start();

    // Wait for all threads to finish
    try {
        customer1.join();
        customer2.join();
        customer3.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    System.out.println("Final account balance: " + account.getBalance());
}
}

```

8. Dining Philosophers Problem Implement the Dining Philosophers Problem using threads in Java. Use synchronization mechanisms to prevent deadlock and starvation.

```

import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Philosopher implements Runnable {
    private final int id;
    private final Lock leftFork;
    private final Lock rightFork;

    public Philosopher(int id, Lock leftFork, Lock rightFork) {
        this.id = id;
        this.leftFork = leftFork;
        this.rightFork = rightFork;
    }

    // Simulate philosopher thinking
    private void think() throws InterruptedException {
        System.out.println("Philosopher " + id + " is thinking.");
        Thread.sleep((int) (Math.random() * 1000)); // Random think time
    }

    // Simulate philosopher eating
    private void eat() throws InterruptedException {
        System.out.println("Philosopher " + id + " is eating.");
        Thread.sleep((int) (Math.random() * 1000)); // Random eat time
    }

    // Pick up both forks (with proper lock order to prevent deadlock)
    private void pickUpForks() {
        leftFork.lock();
        rightFork.lock();
        System.out.println("Philosopher " + id + " picked up both forks.");
    }
}

```

```

// Put down both forks
private void putDownForks() {
    leftFork.unlock();
    rightFork.unlock();
    System.out.println("Philosopher " + id + " put down both forks.");
}

@Override
public void run() {
    try {
        while (true) {
            think(); // Philosopher is thinking
            pickUpForks(); // Philosopher tries to pick up forks
            eat(); // Philosopher is eating
            putDownForks(); // Philosopher puts down forks
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}

public class DiningPhilosophers {
    public static void main(String[] args) {
        int numPhilosophers = 5;
        Philosopher[] philosophers = new Philosopher[numPhilosophers];
        Lock[] forks = new ReentrantLock[numPhilosophers];

        // Initialize forks as locks
        for (int i = 0; i < numPhilosophers; i++) {
            forks[i] = new ReentrantLock();
        }

        // Initialize philosophers and assign them forks
        for (int i = 0; i < numPhilosophers; i++) {
            Lock leftFork = forks[i];
            Lock rightFork = forks[(i + 1) % numPhilosophers];

            // To avoid deadlock, the last philosopher picks up the right fork first
            if (i == numPhilosophers - 1) {
                philosophers[i] = new Philosopher(i, rightFork, leftFork);
            } else {
                philosophers[i] = new Philosopher(i, leftFork, rightFork);
            }

            // Start philosopher threads
            new Thread(philosophers[i], "Philosopher-" + i).start();
        }
    }
}

```

9. Callable and Future Create a thread pool that calculates the square of numbers using the Callable interface and returns results using Future. The main thread should wait for all threads to complete and print the results.

```
import java.util.ArrayList;

import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

// Callable task to compute the square of a number
class SquareCalculator implements Callable<Integer> {
    private final int number;

    public SquareCalculator(int number) {
        this.number = number;
    }

    @Override
    public Integer call() {
        System.out.println("Calculating the square of: " + number);
        return number * number;
    }
}

public class CallableFutureExample {
    public static void main(String[] args) {
        // List of numbers to calculate squares for
        List<Integer> numbers = List.of(2, 4, 6, 8, 10);

        // Create a thread pool with a fixed number of threads
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // List to hold Future objects
        List<Future<Integer>> futureResults = new ArrayList<>();

        // Submit tasks to the thread pool and collect Future objects
        for (int number : numbers) {
            Callable<Integer> task = new SquareCalculator(number);
            Future<Integer> future = executor.submit(task);
            futureResults.add(future);
        }

        // Wait for all tasks to complete and print results
        for (Future<Integer> future : futureResults) {
            try {
                // Get the result of the computation
                Integer result = future.get();
                System.out.println("Result: " + result);
            } catch (InterruptedException | ExecutionException e) {
                e.printStackTrace();
            }
        }

        // Shutdown the executor service
    }
}
```

```

        executor.shutdown();
    }
}

```

10. Multithreaded Sorting Implement a merge sort algorithm that sorts an array using multiple threads. Split the array into subarrays and sort them in parallel, then merge the results.

```

import java.util.Arrays;

import java.util.concurrent.RecursiveTask;
import java.util.concurrent.ForkJoinPool;

public class MultiithreadedMergeSort {

    // Merge two sorted subarrays
    private static void merge(int[] array, int[] left, int[] right) {
        int i = 0, j = 0, k = 0;

        // Merge left and right arrays
        while (i < left.length && j < right.length) {
            if (left[i] < right[j]) {
                array[k++] = left[i++];
            } else {
                array[k++] = right[j++];
            }
        }

        // Copy remaining elements from left array
        while (i < left.length) {
            array[k++] = left[i++];
        }

        // Copy remaining elements from right array
        while (j < right.length) {
            array[k++] = right[j++];
        }
    }

    // Parallel merge sort using ForkJoin framework
    static class ParallelMergeSortTask extends RecursiveTask<int[]> {
        private int[] array;

        public ParallelMergeSortTask(int[] array) {
            this.array = array;
        }

        @Override
        protected int[] compute() {
            if (array.length <= 1) {
                return array;
            }

            int mid = array.length / 2;

            // Split the array into left and right halves
            int[] left = Arrays.copyOfRange(array, 0, mid);
            int[] right = Arrays.copyOfRange(array, mid, array.length);

```

```

        // Sort both halves in parallel
        ParallelMergeSortTask leftTask = new ParallelMergeSortTask(left);
        ParallelMergeSortTask rightTask = new ParallelMergeSortTask(right);

        leftTask.fork(); // Fork the left task
        right = rightTask.compute(); // Compute right half
        left = leftTask.join(); // Wait for left task to finish

        // Merge sorted halves
        merge(array, left, right);

        return array;
    }
}

public static void main(String[] args) {
    int[] array = { 38, 27, 43, 3, 9, 82, 10 };

    // Create ForkJoinPool for parallel execution
    ForkJoinPool pool = new ForkJoinPool();

    // Invoke the merge sort task
    ParallelMergeSortTask task = new ParallelMergeSortTask(array);
    pool.invoke(task);

    // Print sorted array
    System.out.println("Sorted Array: " + Arrays.toString(array));
}
}

```